Getting Started with snowflakeR

`snowflakeR` provides a native R interface to the Snowflake ML platform. This vignette covers installation, connecting to Snowflake, running queries, and using the DBI integration.

Installation

Install the development version from GitHub:

```
# install.packages("pak")
pak::pak("Snowflake-Labs/snowflakeR")
```

`snowflakeR` requires a Python environment with `snowflake-ml-python`. The package includes a helper to set this up:

```
sfr_install_python_deps()
```

Or create the environment manually:

```
conda create -n r-snowflakeR python=3.11 -y
conda activate r-snowflakeR
pip install snowflake-ml-python snowflake-snowpark-python
```

Connecting to Snowflake

Option A: connections.toml (recommended)

If you have a `~/.snowflake/connections.toml` file (the Snowflake standard), `snowflakeR` reads it automatically:

```
library(snowflakeR)

# Default connection from connections.toml
conn <- sfr_connect()

# Named connection
conn <- sfr_connect(name = "production")
```

Option B: Explicit parameters

```
conn <- sfr_connect(
  account   = "xy12345.us-east-1",
  user      = "MYUSER",
  warehouse = "COMPUTE_WH",
  database  = "MY_DB",
  schema    = "MY_SCHEMA",
  authenticator = "externalbrowser"
)
```

Option C: Key-pair authentication

```
conn <- sfr_connect(
  account          = "xy12345.us-east-1",
  user             = "MYUSER",
  private_key_file = "~/.snowflake/rsa_key.p8"
)
```

Option D: Snowflake Workspace Notebooks

In Workspace Notebooks, `sfr_connect()` automatically detects the active session – no credentials needed:

```
conn <- sfr_connect()  # Auto-detects Workspace session
```

Checking the connection

```
# Print connection details
conn

# Check connection status
sfr_status(conn)

# Switch warehouse or schema
sfr_use(conn, warehouse = "ML_WH", schema = "FEATURES")
```

Running Queries

SQL queries

```
# Return results as a data.frame
result <- sfr_query(conn, "SELECT CURRENT_TIMESTAMP() AS now, CURRENT_USER() AS user")
result

# DDL/DML (no result set)
sfr_execute(conn, "CREATE TABLE IF NOT EXISTS test_table (id INT, name STRING)")
```

Reading and writing tables

```
# List tables
sfr_list_tables(conn)

# Check if a table exists
sfr_table_exists(conn, "MY_TABLE")

# Read a table into R
df <- sfr_read_table(conn, "MY_TABLE")

# Write a data.frame to Snowflake
sfr_write_table(conn, "NEW_TABLE", mtcars, overwrite = TRUE)

# Describe a table's columns
sfr_list_fields(conn, "MY_TABLE")
```

DBI / dbplyr Integration

For full DBI compliance (`dbGetQuery`, `dbWriteTable`, `dbListTables`, etc.) and `dbplyr` integration, use the companion **RSnowflake** package. You can bridge from an `sfr_connection` to a DBI connection with `sfr_dbi_connection()`:

```
# Obtain an RSnowflake DBI connection (requires RSnowflake to be installed)
dbi_con <- sfr_dbi_connection(conn)

library(DBI)
DBI::dbGetQuery(dbi_con, "SELECT 1 AS x")
DBI::dbListTables(dbi_con)
DBI::dbExistsTable(dbi_con, "MY_TABLE")
DBI::dbReadTable(dbi_con, "MY_TABLE")
DBI::dbWriteTable(dbi_con, "NEW_TABLE", mtcars, overwrite = TRUE)
```

This also enables `dbplyr` for dplyr-style data manipulation:

```
library(dplyr)
library(dbplyr)

tbl(dbi_con, "MY_TABLE") |>
  filter(status == "active") |>
  group_by(category) |>
  summarise(n = n(), avg_value = mean(value, na.rm = TRUE)) |>
  collect()
```

Alternatively, you can connect directly with RSnowflake without `snowflakeR`:

```
library(DBI)
library(RSnowflake)
con <- dbConnect(Snowflake(), name = "my_profile")
```

Disconnecting

```
sfr_disconnect(conn)
```

Next steps

- **Model Registry**: See `vignette("model-registry")` for logging and deploying R models to Snowflake.
- **Feature Store**: See `vignette("feature-store")` for managing features and generating training data.